Virtual Machines & RBAC with Terraform

Provision a virtual machine and its governance — least-privilege access, required tags, disk encryption, and a budget alert — entirely as code with Terraform.

Exam domains	Manage Azure identities & governance; Deploy & manage compute resources; Monitor & maintain (cost)
Tooling	Terraform (azurerm provider) + Azure CLI for login. Standard four-file layout: main.tf, variables.tf, outputs.tf, terraform.tfvars.
Prerequisites	An Azure subscription with Owner or User Access Administrator rights; Terraform and the Azure CLI installed locally

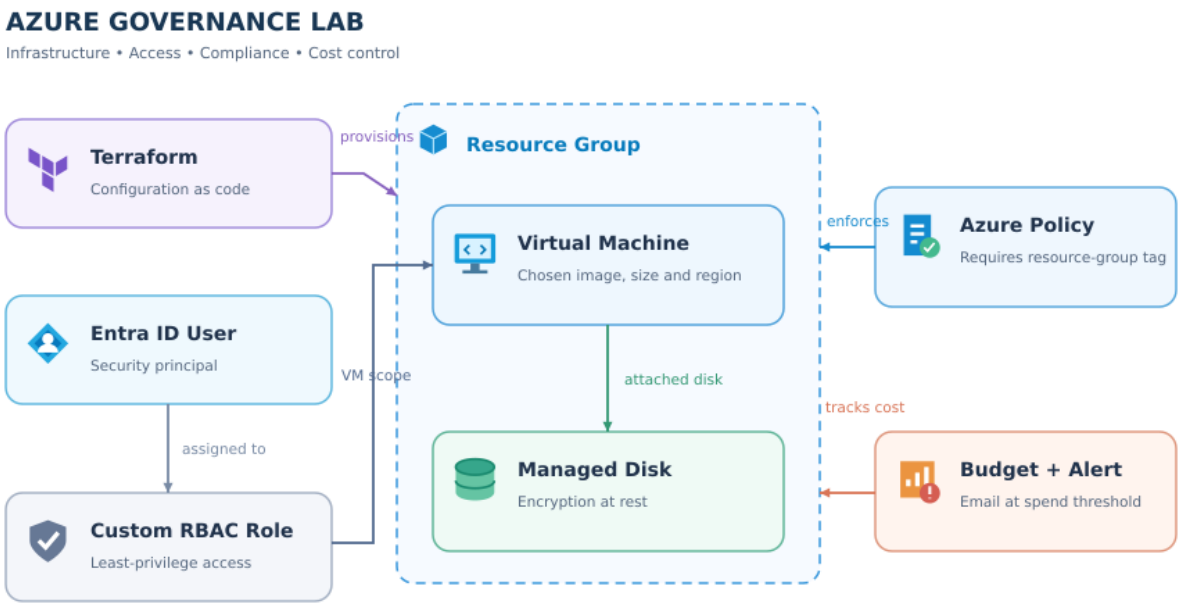
The Business Problem This Lab Solves

Creating a virtual machine in Azure is straightforward. The bigger challenge is making sure it is secure, properly managed, and stays within budget. Who should have access to it? How do we stop users from granting more permissions than they need? And how do we avoid paying for a forgotten VM that has been running all weekend?

This lab explores those questions through a practical scenario. You'll take on the role of an Azure administrator setting up a virtual machine with controls that an auditor, security team, or finance team would expect to see. These include a managed identity, a role with limited permissions, a policy that enforces tagging and disk encryption, and a budget alert to help you monitor spending.

You'll build the setup using Terraform, so you can see how the resources and their settings are defined in code. This makes the environment easier to review, repeat, and update. The aim is to move beyond simply getting a VM running and understand how to manage it responsibly.

Architecture



Terraform provisions everything: a VM in a resource group, a custom least-privilege RBAC role scoped to it, an Azure Policy assignment enforcing tags, customer-managed disk encryption, and a budget alert that watches spend.

Objectives

In this lab, I'll use Terraform to set up an Azure environment with access controls, disk encryption, and spending alerts. I'll create:

- A Terraform project organised into four files, allowing me to deploy the environment with a single `terraform apply`.
- A virtual machine with my chosen size, operating system image, and Azure region.
- A custom RBAC role assigned only to that VM, limiting what the assigned user or identity can do.
- An Azure Policy assignment that requires a specific tag on the resource group.
- Disk encryption using a customer-managed key stored in Azure Key Vault and linked through a disk encryption set.
- A budget for the resource group, with an email alert when spending reaches a set threshold.

Concepts Explained

Infrastructure as Code and Terraform

Infrastructure as Code (IaC) lets me define cloud resources in configuration files instead of creating them manually through the Azure portal. With Terraform, I describe what I want to build, and it works out how to create it.
Terraform shows me a plan before making changes and uses a state file to track the resources it manages. When I finish the lab, I can use terraform destroy to remove them. The azurerm provider connects Terraform to Azure so it can manage those resources.

The Four-File Project Layout

I'll organise this project into four files to keep the configuration easy to follow:

- **main.tf:** Contains the provider configuration and the resources
- **variables.tf:** Defines the inputs, including their types and any default values.
- **terraform.tfvars:** Supplies the values I want to use for those inputs.
- **outputs.tf:** Displays useful information, such as resource IDs, after deployment.

Terraform does not require this exact layout, but it helps me keep the project organised and reuse it with different settings.

Virtual Machine, Size, and Region

A virtual machine is a computer hosted in Azure. I'll choose its size, operating system image, and region based on what the lab needs.
The size determines its CPU and memory and affects its cost. The image determines the operating system, while the region determines where Azure hosts it. I'll define these choices in Terraform so they are easy to review and change.

Role-Based Access Control (RBAC)

RBAC controls **who can do what and where** in Azure. It brings together three things: a user, group, or identity; a role that defines the permitted actions; and a scope where those permissions apply.
In this lab, I'll create a custom role and assign it only to the virtual machine. This lets me practise least privilege: giving an identity only the access it needs to complete its task.

Azure Policy and Tags

Azure Policy helps enforce rules for how resources are configured. Depending on the policy, it can flag settings that do not meet a requirement, block a deployment, or help correct existing resources.
Tags are labels, such as CostCenter = Marketing, that help organise resources and track costs. In this lab, I'll assign a policy that requires a specific tag on the resource group.
RBAC controls the actions someone can perform. Azure Policy checks whether the resulting configuration meets the rules.

Budgets and Cost Alerts

An Azure budget helps me track spending against a limit I choose. I can configure an email alert when actual or forecast spending reaches a percentage of that budget.
For this lab, I'll create a resource-group budget and set a spending threshold for the alert. **The alert will notify me, but it will not stop resources or prevent further charges.**

Step 1 — Create the Empty Files

Open a terminal in the folder where this lab will live, then run the one-liner for your operating system. It creates the four standard Terraform files

Windows (PowerShell)

PowerShell — run in your target folder

```
New-Item -ItemType File main.tf, variables.tf, outputs.tf, terraform.tfvars
```

macOS / Linux / WSL (bash)

bash — run in your target folder

```
touch main.tf variables.tf outputs.tf terraform.tfvars
```

Step 2 — Fill In the Files

variables.tf — declare the inputs

Declaring inputs here (instead of hardcoding) is what lets you reuse this config across environments.

variables.tf

```
#variables
variable "location" {
    description = "This will be Azure regions for all resources"
type = string
}

variable "prefix" {
    description = "this is suffix that will resource names uniques"
    type = string
}
variable "email_alert" {
description = "Email Address that will receive budget alert"
type = string
}
variable "operator_object_id" {
  description = "This is Entra Object ID of the user to grant VM read and restart access"
  type = string
}
```

terraform.tfvars — set your values

Replace the placeholder object ID and email with yours. This is the only file you change per environment.

```
#the actual values of the variables
location = "eastus"
prefix = "gh-lab099"
email_alert = "you@outlook.com"
operator_object_id = "0000-000"
```

main.tf (part 1) — providers

The provider block goes at the top of main.tf. After pasting it, initialize.

main.tf

```
# Configure the Azure provider
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0.2"
    }
  }

  required_version = ">= 1.1.0"
}

provider "azurerm" {
  features {}
}
```

1. **Initialize.** Run terraform init. Terraform downloads the azurerm provider and prepares the folder.

main.tf (part 2) — the resource group

Everything else is created inside this group, and it carries the tag the policy will require.

main.tf (append)

```
# +rg
resource "azurerm_resource_group" "rg" {
  name     = "${var.prefix}-rg-vm-rbac"
  location = var.location
  tags = {
    Project = "${var.prefix}-rbac"
  }
}
```

main.tf (part 3) — network and the virtual machine

The VM needs a VNet, subnet, and NIC; a B-series size keeps cost low.

main.tf (append)

```
# +vnet +subnet
resource "azurerm_virtual_network" "vnet" {
  name                = "${var.prefix}-vnet"
  address_space       = ["10.0.0.0/16"]
```

```
  location            = var.location
  resource_group_name = azurerm_resource_group.rg.name
  tags = {
  Project    = "${var.prefix}-rbac"
  CostCenter = var.prefix
}
}
# +subnet
resource "azurerm_subnet" "subnet" {
  name                 = "${var.prefix}-subnet"
  resource_group_name  = azurerm_resource_group.rg.name
  address_prefixes     = ["10.0.1.0/24"]
  virtual_network_name = azurerm_virtual_network.vnet.name


}
# +NIC
resource "azurerm_network_interface" "nic" {
  name                = "${var.prefix}-nic"
  location            = azurerm_virtual_network.vnet.location
  resource_group_name = azurerm_resource_group.rg.name
  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.subnet.id
    private_ip_address_allocation = "Dynamic" #let azure assign the ip
  }
tags = {
  CostCenter = var.prefix
}
}
```

```
# +VM itself
resource "azurerm_linux_virtual_machine" "vm" {
  name                  = "${var.prefix}-vm"
  resource_group_name   = azurerm_resource_group.rg.name
  location              = azurerm_resource_group.rg.location
  size                  = "Standard_B1s"
  admin_username        = "azureuser"
  network_interface_ids = [azurerm_network_interface.nic.id]
  admin_ssh_key {
    username   = "azureuser"
    public_key = file(pathexpand("~/.ssh/azure_vsVm_rsa.pub")) # public key
  }
  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }
  source_image_reference { # which OS image to boot
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }
tags = {
  Project    = "${var.prefix}-rbac"
  CostCenter = var.prefix
}
}
```

main.tf (part 4) — custom RBAC role and assignment

A role that can read and restart only this VM, granted to your operator.

main.tf (append)

```
# +custom role: this role allows user to read and restart vm and it only avaiable for vms
resource "azurerm_role_definition" "vm_restart" {
  description = "Read and restart this VM only"
  name        = "${var.prefix}-VM Restart Users"
  scope       = azurerm_linux_virtual_machine.vm.id
  permissions {
    actions = [
      "Microsoft.Compute/virtualMachines/read",
      "Microsoft.Compute/virtualMachines/restart/action"

    ]
    not_actions = []

  }
  assignable_scopes = [azurerm_linux_virtual_machine.vm.id]

}

# + assign the role to user or users
resource "azurerm_role_assignment" "vm_restart" {
    scope = azurerm_linux_virtual_machine.vm.id
    role_definition_id = azurerm_role_definition.vm_restart.role_definition_id
    principal_id = var.operator_object_id #assigned user entra object id

}
```

main.tf (part 5) — enforce tagging with Azure Policy

Assign the built-in tag policy so untagged resources in the group are denied.

main.tf (append)

```
# Look up the built-in policy that requires a tag on resources
data "azurerm_policy_definition" "require_tag" {
  display_name = "Require a tag on resources"
}
# Assign it to the resource group: untagged resources get denied
resource "azurerm_resource_group_policy_assignment" "tags" {
  name                 = "require-costcenter"
  resource_group_id    = azurerm_resource_group.rg.id
  policy_definition_id = data.azurerm_policy_definition.require_tag.id
  parameters = jsonencode({ tagName = { value = "CostCenter" } }) # tag to enforce
}
```

main.tf (part 6) — budget with a cost alert

A monthly budget that emails you at 80% of the threshold.

main.tf (append)

```
# +BUDGET + ALERT WARNS (does not enforce) when spend crosses the threshold
resource "azurerm_consumption_budget_resource_group" "budget" {
    name = "${var.prefix}-budget"
    resource_group_id = azurerm_resource_group.rg.id
    amount = 10 # monthly
    time_grain = "Monthly"
    time_period {
      start_date = "2026-10-01T00:00:00Z"
    }
    notification {
      enabled = true
      threshold = 80    # get email at 80% of GBP10 which is GBP8
      operator = "GreaterThanOrEqualTo"
      contact_emails = [var.email_alert]
    }

}
```

outputs.tf — expose key values

Outputs print after apply and let other configs reference these resources without hardcoding names.

outputs.tf

```
output "resource_group_name" {
  value = azurerm_resource_group.rg.name
}

output "vm_name" {
  value = azurerm_linux_virtual_machine.vm.name
}

output "vm_id" {
  value = azurerm_linux_virtual_machine.vm.id
}

output "custom_role_name" {
  value = azurerm_role_definition.vm_restart.name
}
```

Step 3 — Plan and apply

1. **Review the plan.** Run terraform plan. Every resource should show as "+ create." This is your chance to catch mistakes before anything is built.
2. **Apply.** Run terraform apply and type yes. Terraform reads terraform.tfvars automatically and creates everything in dependency order, then prints your outputs.
3. **Note the state.** Terraform records what it built in terraform.tfstate. Never edit that file by hand — it is how Terraform knows what exists.

How to Confirm It Works

- Check that terraform apply completes successfully and displays the outputs. Run terraform state list to see the resources Terraform manages.
- Sign in as the operator. Restarting the VM should work, while deleting it should be denied—provided no other assigned role grants delete access.
- Test the tagging policy by attempting to create a resource group without the required tag. Creation should be blocked when the policy is assigned at subscription scope.
- In the Azure portal, confirm that the VM's disk uses the customer-managed encryption key and that the budget's email alert is configured at 80%.

Clean Up to Avoid Charges

When you finish the lab, run terraform destroy. Review the resources listed for deletion, then type yes to confirm. Terraform removes the resources it manages in dependency order, saving you from deleting them individually in the Azure portal. Afterwards, check for anything left behind, including resources created manually. Some items, such as a Key Vault with purge protection enabled, may prevent a complete cleanup

How This Shows Up

Exam topic	What the question looks like	Why building this makes it easy
Least-privilege RBAC	"Which role grants the minimum access to restart a VM?"	You authored exactly that custom role and saw which actions it needed.
RBAC vs Policy	"Block creation of untagged resources — RBAC or Policy?"	You assigned the tag policy and know RBAC could never do that.
Cost management	"What happens when a budget threshold is reached?"	You configured the notification and know it alerts, not enforces.
IaC concepts	"Why use variables/tfvars / what does a plan show?"	You split inputs into tfvars and ran plan, apply, and destroy.